queries.sql

```sql
/* ==============================
   DDL STATEMENTS - SINGLE PROGRAM
   ============================== */

/* CREATE TABLE */
CREATE TABLE student (
    sid NUMBER,
    sname VARCHAR2(20)
);

/* ALTER TABLE - Add a new column */
ALTER TABLE student
ADD age NUMBER;

/* INSERT RECORDS */
INSERT INTO student VALUES (1, 'Ravi', 20);
INSERT INTO student VALUES (2, 'Anu', 21);

/* DISPLAY RECORDS */
SELECT * FROM student;

/* TRUNCATE TABLE - Remove all records */
TRUNCATE TABLE student;

/* DROP TABLE - Delete table completely */
DROP TABLE student;
```

Output:

```
       SID SNAME                AGE
---------- -------------------- ----------
         1 Ravi                 20
         2 Anu                  21
DROP TABLE student
           *
ERROR at line 1:
ORA-00942: table or view "sandbox_user"."STUDENT" does not exist
```

OneCompiler Pricing Learn

queries.sql 44c5d4q6x

```sql
/* ==============================
   PARENT TABLE
   ============================== */

CREATE TABLE department (
    dept_id NUMBER PRIMARY KEY,          -- Primary Key
    dept_name VARCHAR2(20) UNIQUE         -- Unique constraint
);

/* ==============================
   CHILD TABLE
   ============================== */

CREATE TABLE employee (
    emp_id NUMBER PRIMARY KEY,            -- Primary Key
    emp_name VARCHAR2(20) NOT NULL,        -- NOT NULL constraint
    age NUMBER CHECK (age >= 18),          -- CHECK constraint
    salary NUMBER CHECK (salary > 5000),   -- CHECK constraint
    dept_id NUMBER,
    CONSTRAINT emp_dept_fk FOREIGN KEY (dept_id)
    REFERENCES department(dept_id)         -- Foreign Key
);

/* ==============================
   INSERT VALUES
   ============================== */

INSERT INTO department VALUES (1, 'IT');
INSERT INTO department VALUES (2, 'CSE');

INSERT INTO employee VALUES (101, 'Ravi', 22, 10000, 1);
INSERT INTO employee VALUES (102, 'Anu', 25, 12000, 2);

/* ==============================
   IS (NULL) CONDITION
   ============================== */

SELECT * FROM employee
WHERE dept_id IS NOT NULL;
```

Output: 138 ms

```
    EMP_ID EMP_NAME             AGE     SALARY    DEPT_ID
---------- -------------------- ---------- ---------- ----------
       101 Ravi                 22  10000          1
       102 Anu                  25  12000          2

    EMP_ID EMP_NAME             AGE     SALARY    DEPT_ID
---------- -------------------- ---------- ---------- ----------
       101 Ravi                 22  10000          1
       102 Anu                  25  12000          2
```

queries.sql 44c5d4q6x

```sql
/* Create table */
CREATE TABLE employee (
    emp_id NUMBER,
    emp_name VARCHAR2(20),
    salary NUMBER,
    doj DATE
);

/* Insert values */
INSERT INTO employee VALUES (1, 'ravi', 12000, SYSDATE);
INSERT INTO employee VALUES (2, 'anu', 15000, SYSDATE);
INSERT INTO employee VALUES (3, 'kumar', 10000, SYSDATE);

/* STRING FUNCTIONS */
SELECT UPPER(emp_name) FROM employee;
SELECT LENGTH(emp_name) FROM employee;

/* NUMBER FUNCTIONS */
SELECT MAX(salary), MIN(salary) FROM employee;
SELECT ROUND(AVG(salary), 2) FROM employee;

/* DATE FUNCTIONS */
SELECT SYSDATE FROM dual;
SELECT ADD_MONTHS(doj, 6) FROM employee;

/* AGGREGATE FUNCTION */
SELECT COUNT(emp_id) FROM employee;
```

Output:

```
UPPER(EMP_NAME)
-------------------
RAVI
ANU
KUMAR

LENGTH(EMP_NAME)
----------------
               4
               3
               5

MAX(SALARY) MIN(SALARY)
---------- ----------
     15000      10000

ROUND(AVG(SALARY),2)
--------------------
           12333.33

SYSDATE
------------------
30-JAN-26

ADD_MONTHS(DOJ,6)
------------------
30-JUL-26
30-JUL-26
30-JUL-26

COUNT(EMP_ID)
-------------
            3

COUNT(EMP_ID)
-------------
            3
```

```sql
/* ==============================
   CREATE TABLES
   ============================== */

CREATE TABLE department (
    dept_id NUMBER,
    dept_name VARCHAR2(20)
);

CREATE TABLE employee (
    emp_id NUMBER,
    emp_name VARCHAR2(20),
    dept_id NUMBER
);

/* ==============================
   INSERT VALUES
   ============================== */

INSERT INTO department VALUES (1, 'IT');
INSERT INTO department VALUES (2, 'CSE');
INSERT INTO department VALUES (3, 'ECE');

INSERT INTO employee VALUES (101, 'Ravi', 1);
INSERT INTO employee VALUES (102, 'Anu', 2);
INSERT INTO employee VALUES (103, 'Kumar', NULL);

/* ==============================
   INNER JOIN
   ============================== */

SELECT e.emp_name, d.dept_name
FROM employee e
INNER JOIN department d
ON e.dept_id = d.dept_id;

/* ==============================
   LEFT OUTER JOIN
   ============================== */

SELECT e.emp_name, d.dept_name
FROM employee e
LEFT OUTER JOIN department d
ON e.dept_id = d.dept_id;

/* ==============================
   RIGHT OUTER JOIN
   ============================== */

SELECT e.emp_name, d.dept_name
FROM employee e
RIGHT OUTER JOIN department d
ON e.dept_id = d.dept_id;

/* ==============================
   FULL OUTER JOIN
   ============================== */

SELECT e.emp_name, d.dept_name
FROM employee e
FULL OUTER JOIN department d
ON e.dept_id = d.dept_id;

/* ==============================
   CROSS JOIN
   ============================== */

SELECT e.emp_name, d.dept_name
FROM employee e
CROSS JOIN department d;
```

Output: 150 ms

```
EMP_NAME         DEPT_NAME
-------------------- --------------------
Ravi             IT
Anu          CSE

EMP_NAME         DEPT_NAME
-------------------- --------------------
Ravi             IT
Anu          CSE
Kumar

EMP_NAME         DEPT_NAME
-------------------- --------------------
Ravi             IT
Anu          CSE
             ECE

EMP_NAME         DEPT_NAME
-------------------- --------------------
Ravi             IT
Anu          CSE
             ECE
Kumar

EMP_NAME         DEPT_NAME
-------------------- --------------------
Ravi             IT
Ravi             CSE
Ravi             ECE
Anu          IT
Anu          CSE
Anu          ECE
Kumar            IT
Kumar            CSE
Kumar            ECE

EMP_NAME         DEPT_NAME
-------------------- --------------------
Ravi             IT
Ravi             CSE
Ravi             ECE
Anu          IT
Anu          CSE
Anu          ECE
Kumar            IT
Kumar            CSE
Kumar            ECE
```

```sql
/* ==============================
   CREATE TABLE
   ============================== */

CREATE TABLE employee (
    emp_id NUMBER,
    emp_name VARCHAR2(20),
    salary NUMBER,
    dept_id NUMBER
);

/* ==============================
   INSERT VALUES
   ============================== */

INSERT INTO employee VALUES (1, 'Ravi', 12000, 1);
INSERT INTO employee VALUES (2, 'Anu', 15000, 2);
INSERT INTO employee VALUES (3, 'Kumar', 10000, 1);
INSERT INTO employee VALUES (4, 'Meena', 18000, 3);

/* ==============================
   SUBQUERY EXAMPLES
   ============================== */

/* 1. Employee with salary greater than average salary */
SELECT emp_name
FROM employee
WHERE salary > (
    SELECT AVG(salary) FROM employee
);

/* 2. Employees working in the same department as Ravi */
SELECT emp_name
FROM employee
WHERE dept_id = (
    SELECT dept_id
    FROM employee
    WHERE emp_name = 'Ravi'
);

/* 3. Employee with maximum salary */
SELECT emp_name
FROM employee
WHERE salary = (
    SELECT MAX(salary) FROM employee
);
```

Output:

```
EMP_NAME
--------------------
Anu
Meena

EMP_NAME
--------------------
Ravi
Kumar

EMP_NAME
--------------------
Meena

EMP_NAME
--------------------
Meena
```

2:53

PL/SQL Online Editor...
onecompiler.com

OneCompiler Pricing Learn Code Deploy More A

queries.sql 44bzvy3kx AI PLSQL RUN

```
DECLARE
    num1 NUMBER := 10;
    num2 NUMBER := 20;
BEGIN
    IF num1 < num2 THEN
        DBMS_OUTPUT.PUT_LINE('num1 is smaller than num2');
    ELSIF num1 = num2 THEN
        DBMS_OUTPUT.PUT_LINE('num1 and num2 are equal');
    ELSE
        DBMS_OUTPUT.PUT_LINE('num2 is smaller than num1');
    END IF;

    DBMS_OUTPUT.PUT_LINE('after else if ladder');
END;
/
```

STDIN

Input for the program (Optional)

Output: 102 ms

```
num1 is smaller than num2
after else if ladder
```

queries.sql 44c5dz8gr AI PLSQL RUN

```
DECLARE
    a NUMBER := 1;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Program started');

    LOOP
        DBMS_OUTPUT.PUT_LINE(a);
        a := a + 1;

        EXIT WHEN a > 5;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Program completed');
END;
/
```

STDIN

Input for the program (Optional)

Output: 113 ms

```
Program started
1
2
3
4
5
Program completed
```

queries.sql

44fw9ggvw

AI PLSQL RUN

```sql
DECLARE
    TYPE namesarray IS VARRAY(5) OF VARCHAR2(10);
    TYPE grades IS VARRAY(5) OF INTEGER;

    names namesarray;
    marks grades;
    total INTEGER;

BEGIN
    names := namesarray('Kavita','Pritam','Ayan','Rishav','Aziz');
    marks := grades(98,97,78,87,92);

    total := names.COUNT;

    dbms_output.put_line('Total ' || total || ' Students');

    FOR i IN 1..total LOOP
        dbms_output.put_line('Student: ' || names(i) ||
                             ' Marks: ' || marks(i));
    END LOOP;
END;
/
```

STDIN

Input for the program (Optional)

Output:

```
Total 5 Students
Student: Kavita Marks: 98
Student: Pritam Marks: 97
Student: Ayan Marks: 78
Student: Rishav Marks: 87
Student: Aziz Marks: 92
```

OneCompiler Pricing Learn Code

queries.sql 44fw9ggvw AI

```sql
/* ================================
   CREATE TABLES
   ================================ */

CREATE TABLE employees (
    employee_id NUMBER,
    name        VARCHAR2(50),
    salary      NUMBER
);

CREATE TABLE emp_log (
    emp_id      NUMBER,
    action_date DATE
);

/* ================================
   BEFORE INSERT TRIGGER
   ================================ */

CREATE OR REPLACE TRIGGER before_insert_emp
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    -- Prevent negative salary
    IF :NEW.salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
        'Salary cannot be negative');
    END IF;
END;
/

/* ================================
   AFTER INSERT TRIGGER
   ================================ */

CREATE OR REPLACE TRIGGER after_insert_emp
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO emp_log(emp_id, action_date)
    VALUES(:NEW.employee_id, SYSDATE);
END;
/

/* ================================
   INSERT DATA
   ================================ */

INSERT INTO employees VALUES (1, 'Ravi', 25000);

INSERT INTO employees VALUES (2, 'Anu', -5000);  -- Will raise error

/* ================================
   VIEW DATA
   ================================ */

SELECT * FROM employees;
SELECT * FROM emp_log;
```

```
SQL> CREATE TABLE employees (
  2       employee_id NUMBER,
  3       name        VARCHAR2(50),
  4       salary      NUMBER
  5  );

Table created.

SQL>
SQL> CREATE TABLE emp_log (
  2       emp_id      NUMBER,
  3       action_date DATE
  4  );

Table created.

SQL>
SQL> /* ================================
SQL>    BEFORE INSERT TRIGGER
SQL>    ================================ */
SQL>
SQL> CREATE OR REPLACE TRIGGER before_insert_emp
  2  BEFORE INSERT ON employees
  3  FOR EACH ROW
  4  BEGIN
  5      -- Prevent negative salary
  6      IF :NEW.salary < 0 THEN
  7          RAISE_APPLICATION_ERROR(-20001,
  8          'Salary cannot be negative');
  9      END IF;
 10  END;
 11  /

Trigger created.
```

```
SQL>
SQL> /* ================================
SQL>    AFTER INSERT TRIGGER
SQL>    ================================ */
SQL>
SQL> CREATE OR REPLACE TRIGGER after_insert_emp
  2  AFTER INSERT ON employees
  3  FOR EACH ROW
  4  BEGIN
  5      INSERT INTO emp_log(emp_id, action_date)
  6      VALUES(:NEW.employee_id, SYSDATE);
  7  END;
  8  /

Trigger created.

SQL>
SQL> /* ================================
SQL>    INSERT DATA
SQL>    ================================ */
SQL>
SQL> INSERT INTO employees VALUES (1, 'Ravi', 25000);

1 row created.

SQL> INSERT INTO employees VALUES (2, 'Anu', -5000);
INSERT INTO employees VALUES (2, 'Anu', -5000)
            *
ERROR at line 1:
ORA-20001: Salary cannot be negative
ORA-06512: at "SYSTEM.BEFORE_INSERT_EMP", line 4
ORA-04088: error during execution of trigger 'SYSTEM.BEFORE_INSERT_EMP'


SQL> SELECT * FROM employees;

EMPLOYEE_ID NAME                                                 SALARY
----------- -------------------------------------------------- ----------
          1 Ravi                                                   25000
```

```
SQL> SELECT * FROM employees;

EMPLOYEE_ID NAME                                                 SALARY
----------- -------------------------------------------------- ----------
          1 Ravi                                                    25000

SQL> SELECT * FROM emp_log;

    EMP_ID ACTION_DA
---------- ---------
         1 10-MAR-26

SQL> |
```

```oracle_plsql
/* ====================================
   PL/SQL TRIGGERS FOR DATA VALIDATION
   ==================================== */

/* 1. CREATE EMPLOYEE TABLE */

CREATE TABLE employees (
    empno  NUMBER(5) PRIMARY KEY,
    name   VARCHAR2(20),
    basic  NUMBER(10),
    hra    NUMBER(10),
    da     NUMBER(10)
);

/* 2. INSERT SAMPLE RECORDS */

INSERT INTO employees VALUES (101, 'Ravi', 25000, 5000, 3000);
INSERT INTO employees VALUES (102, 'Anu', 30000, 6000, 3500);

/* 3. CREATE BACKUP TABLE */

CREATE TABLE backup (
    empno NUMBER(5),
    old_basic NUMBER(10),
    action_date DATE
);

/* 4. BEFORE UPDATE TRIGGER (VALIDATION) */

CREATE OR REPLACE TRIGGER trg_before_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF :NEW.basic < 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
        'Salary cannot be negative');
    END IF;
END;
/

/* 5. AFTER UPDATE TRIGGER (BACKUP) */

CREATE OR REPLACE TRIGGER trg_after_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO backup(empno, old_basic, action_date)
    VALUES(:OLD.empno, :OLD.basic, SYSDATE);
END;
/

/* 6. BEFORE DELETE TRIGGER (VALIDATION) */

CREATE OR REPLACE TRIGGER trg_before_delete
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    IF :OLD.basic > 50000 THEN
        RAISE_APPLICATION_ERROR(-20002,
        'High salary employee cannot be deleted');
    END IF;
END;
/

/* 7. UPDATE SALARY (VALID) */

UPDATE employees
SET basic = 28000
WHERE empno = 101;

/* 8. UPDATE SALARY (INVALID - WILL SHOW ERROR) */

UPDATE employees
SET basic = -1000
WHERE empno = 102;

/* 9. DELETE RECORD (CONDITION CHECK) */

DELETE FROM employees
WHERE empno = 101;

/* 10. VIEW RECORDS */

SELECT * FROM employees;
SELECT * FROM backup;
```

```
SQL> CREATE TABLE employees (
  2       empno  NUMBER(5) PRIMARY KEY,
  3       name   VARCHAR2(20),
  4       basic  NUMBER(10),
  5       hra    NUMBER(10),
  6       da     NUMBER(10)
  7  );

Table created.

SQL>
SQL> INSERT INTO employees VALUES (101, 'Ravi', 25000, 5000, 3000);

1 row created.

SQL> INSERT INTO employees VALUES (102, 'Anu', 30000, 6000, 3500);

1 row created.

SQL> CREATE TABLE backup (
  2       empno NUMBER(5),
  3       old_basic NUMBER(10),
  4       action_date DATE
  5  );

Table created.
```

```
Table created.

SQL> CREATE OR REPLACE TRIGGER trg_before_update
  2  BEFORE UPDATE ON employees
  3  FOR EACH ROW
  4  BEGIN
  5      IF :NEW.basic < 0 THEN
  6          RAISE_APPLICATION_ERROR(-20001,
  7          'Salary cannot be negative');
  8      END IF;
  9  END;
 10  /

Trigger created.

SQL> CREATE OR REPLACE TRIGGER trg_after_update
  2  AFTER UPDATE ON employees
  3  FOR EACH ROW
  4  BEGIN
  5      INSERT INTO backup(empno, old_basic, action_date)
  6      VALUES(:OLD.empno, :OLD.basic, SYSDATE);
  7  END;
  8  /

Trigger created.

SQL>
SQL> /* 6. BEFORE DELETE TRIGGER (VALIDATION) */
SQL>
SQL> CREATE OR REPLACE TRIGGER trg_before_delete
  2  BEFORE DELETE ON employees
  3  FOR EACH ROW
  4  BEGIN
  5      IF :OLD.basic > 50000 THEN
  6          RAISE_APPLICATION_ERROR(-20002,
  7          'High salary employee cannot be deleted');
  8      END IF;
  9  END;
 10  /
```

```
Trigger created.

SQL>
SQL> /* 7. UPDATE SALARY (VALID) */
SQL>
SQL> UPDATE employees
  2  SET basic = 28000
  3  WHERE empno = 101;

1 row updated.

SQL>
SQL> /* 8. UPDATE SALARY (INVALID - WILL SHOW ERROR) */
SQL>
SQL> UPDATE employees
  2  SET basic = -1000
  3  WHERE empno = 102;
UPDATE employees
       *
ERROR at line 1:
ORA-20001: Salary cannot be negative
ORA-06512: at "SYSTEM.TRG_BEFORE_UPDATE", line 3
ORA-04088: error during execution of trigger 'SYSTEM.TRG_BEFORE_UPDATE'


SQL>
SQL> /* 9. DELETE RECORD (CONDITION CHECK) */
SQL>
SQL> DELETE FROM employees
  2  WHERE empno = 101;

1 row deleted.

SQL>
SQL> /* 10. VIEW RECORDS */
SQL>
SQL> SELECT * FROM employees;
```

```
SQL>
SQL> /* 9. DELETE RECORD (CONDITION CHECK) */
SQL>
SQL> DELETE FROM employees
  2  WHERE empno = 101;

1 row deleted.

SQL>
SQL> /* 10. VIEW RECORDS */
SQL>
SQL> SELECT * FROM employees;

     EMPNO NAME                          BASIC        HRA         DA
---------- -------------------- ---------- ---------- ----------
       102 Anu                         30000       6000       3500

SQL> SELECT * FROM backup;

     EMPNO  OLD_BASIC ACTION_DA
---------- ---------- ---------
       101      25000 10-MAR-26
```

OneCompiler

Pricing Learn Code Deploy More

queries.sql 44e8szs4w

AI PLSQL RUN

```sql
SET SERVEROUTPUT ON;

DECLARE
    a NUMBER;

    PROCEDURE squareNum(x IN OUT NUMBER) IS
    BEGIN
        x := x * x;
    END;

BEGIN
    a := 23;

    squareNum(a);

    DBMS_OUTPUT.PUT_LINE('Square of (23): ' || a);
END;
/
```

STDIN

Input for the program (Optional)

Output: 104 ms

```
Square of (23): 529
```

```
Function Add(a As Integer, b As Integer) As
Integer
   Add = a + b
End Function
Private Sub Command1_Click()
   Dim result As Integer
   result = Add(10, 20)
   MsgBox "Sum = " & result
End Sub
```

3:56 pm ✓✓

12th program 1st form query for functions

Edited 3:56 pm ✓✓

```
Private Sub AddNumbers(a As Integer, b As
Integer)
   Dim sum As Integer
   sum = a + b
   MsgBox "Sum = " & sum
End Sub

Private Sub Command1_Click()
   Call AddNumbers(20, 20)
End Sub
```

3:57 pm ✓✓

12th program 2nd form query for procedure

3:57 pm ✓✓